Software Requirements Specification
Restaurant App MVP

Mobile Application Development — Assignment 1

Prepared by: Ayesha

Institution: Abasyn University

Semester: 4th

Document Version: 1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements of the **Restaurant App MVP**, a mobile application built using React Native (Expo). The purpose of this document is to define, in a clear and unambiguous manner, what the application must do so that it can be implemented, tested, and evaluated against a single, agreed specification. This document is intended for the developer (student) and the course instructor evaluating Assignment 1.

1.2 Scope

The Restaurant App MVP is a **frontend-only** mobile application that allows customers to browse a restaurant's menu, search and filter items, add items to a cart, apply promotional discounts, reserve a table, place an order (dine-in or takeaway), and track the status of that order in real time. It also allows a Restaurant Manager to view and manage incoming orders, accept or decline reservations, and manage the menu (add items, edit prices, toggle availability). The application does not use any backend server, external database, or third-party API. All data is either hard-coded as local/mock JavaScript data or persisted on-device using AsyncStorage. State is managed exclusively using React's built-in hooks (useState, useReducer, useContext) — no external state-management library (e.g. Redux) is used.

1.3 Definitions and Acronyms

Term	Definition
MVP	Minimum Viable Product — the smallest working version of the app that satisfies the core requirements.
SRS	Software Requirements Specification — this document.
UML	Unified Modeling Language — used to diagram the system's structure and behaviour.
FR	Functional Requirement — a specific behaviour the system must perform.
NFR	Non-Functional Requirement — a quality attribute the system must satisfy.
AsyncStorage	A React Native local key-value storage system used to persist data on the device.
Expo	A framework and toolchain built around React Native for building mobile apps.
Customer	An end-user of the app who browses the menu, orders food, and books reservations.
Manager	A restaurant staff member who manages orders, reservations, and menu content.
Hook	A React function (e.g. useState, useEffect) that lets function components use React features.

2. Overall Description

2.1 User Roles

The system supports two user roles, distinguished at signup/login time and stored against the authenticated user's session:

- **Customer** — browses the menu, searches/filters items, manages a cart, applies promo codes, makes table reservations, places orders, and tracks order status.
- **Restaurant Manager** — views and updates incoming order statuses, accepts or declines reservations, and manages menu items (add, edit price, toggle availability).

2.2 User Stories

The following user stories describe the core interactions supported by the app, written from the perspective of both roles:

- **US-01:** As a Customer, I want to sign up with my name, email, and password, so that I can access the app with my own account.
- **US-02:** As a Customer, I want to browse the menu grouped by category, so that I can quickly find the type of food I want.
- **US-03:** As a Customer, I want to see which items are marked as a Daily Special, so that I can try the restaurant's featured dishes.
- **US-04:** As a Customer, I want to search the menu by name, so that I can find a specific dish quickly without scrolling.
- **US-05:** As a Customer, I want to add items to a cart and adjust their quantity, so that I can order exactly what I want.
- **US-06:** As a Customer, I want to apply a promo code at checkout, so that I can receive a discount on my order.
- **US-07:** As a Customer, I want to reserve a table for a specific date, time, and party size, so that I don't have to wait when I arrive.
- **US-08:** As a Customer, I want to track my order's status after placing it, so that I know when my food will be ready.
- **US-09:** As a Customer, I want to switch between light and dark theme, so that I can use the app comfortably at any time of day.
- **US-10:** As a Restaurant Manager, I want to see all incoming orders in one dashboard, so that I can manage kitchen workflow efficiently.

2.3 Customer Stories (Self-Interview)

As part of requirements gathering, a short self-interview was conducted, imagining myself as a regular restaurant customer and noting down the needs and frustrations that shaped the following stories:

- **CS-01:** “I usually decide what to eat based on pictures — if the menu has no images or descriptions, I end up guessing.” → The menu must show an image, name, description, and price for every item.
- **CS-02:** “I hate typing a full dish name to search for it — I want results as I type.” → The search bar must filter results live, without needing to press a search button.
- **CS-03:** “When I'm at a restaurant with friends, I always forget what I already added to the cart.” → The cart must clearly show a running item count and total at all times.
- **CS-04:** “I don't like calling ahead to book a table — I'd rather just pick a time slot in an app.” → The reservation screen must let me pick from available time slots directly.
- **CS-05:** “Once I've ordered, I just want to know roughly how long I'll wait, without asking the waiter repeatedly.” → The order tracking screen must show the current status and elapsed time automatically.

2.4 Manager Stories

- **MS-01:** As a Manager, I want to change an order's status (Pending → Preparing → Ready → Served) from a single dashboard, so that customers see accurate progress without me having to inform them manually.
- **MS-02:** As a Manager, I want to accept or decline incoming table reservations, so that I can control seating capacity during busy hours.
- **MS-03:** As a Manager, I want to add new menu items or update prices and availability, so that the menu customers see always reflects what the kitchen can currently serve.

3. Functional Requirements

Each functional requirement below begins with “The system shall” and is grouped by feature area.

3.1 Authentication

ID	Requirement
FR-01	The system shall allow a new user to sign up by providing full name, email, password, confirm password, and role (Customer or Manager).
FR-02	The system shall validate that the entered email follows a standard email format before allowing signup or login.
FR-03	The system shall require a password of at least 8 characters containing at least one digit.
FR-04	The system shall verify that the password and confirm-password fields match before creating an account.

3.2 Menu

ID	Requirement
FR-05	The system shall display all menu items grouped into the categories Starters, Mains, Desserts, and Drinks.
FR-06	The system shall display, for each menu item, its name, description, price, category, image, and availability.
FR-07	The system shall visually mark items flagged as Daily Special with a distinct badge.
FR-08	The system shall visually distinguish unavailable items and disable their Add-to-Cart button.
FR-09	The system shall show a loading indicator while menu data is being prepared and an error state with a Retry option if loading fails.
FR-10	The system shall support pull-to-refresh on the menu list to reload the current data.

3.3 Search

ID	Requirement
FR-11	The system shall allow the customer to search menu items by name using a dedicated search input.
FR-12	The system shall filter search results as the customer types, using a debounce delay of 400ms.
FR-13	The system shall display a clear empty-state message when a search returns no matching items.
FR-14	The system shall maintain a list of the last 5 unique searches performed by the customer.

3.4 Cart

ID	Requirement
FR-15	The system shall allow a customer to add a menu item to the cart from the Menu screen.
FR-16	The system shall allow a customer to increase or decrease the quantity of an item already in the cart.

ID	Requirement
FR-17	The system shall allow a customer to remove an item entirely from the cart.
FR-18	The system shall allow a customer to attach a special instruction/note to a cart item.
FR-19	The system shall allow a customer to apply a valid promo code (WELCOME10 or FEAST20) to receive a percentage discount, and shall display an error message for an invalid code.

3.5 Reservation

ID	Requirement
FR-20	The system shall allow a customer to select a reservation date that is not in the past.
FR-21	The system shall offer reservation time slots between 12:00 and 22:00 and disable slots that are already booked.
FR-22	The system shall require the reservation to be made at least one hour ahead of the selected time slot.
FR-23	The system shall validate the party size to be between 1 and 12 guests.
FR-24	The system shall validate the contact phone number against the Pakistani mobile format (03XX-XXXXXXX).
FR-25	The system shall show a confirmation modal after a reservation is successfully created and allow the customer to view or cancel it from “My Reservations”.

3.6 Orders

ID	Requirement
FR-26	The system shall allow a customer to choose between Dine-in (with table selection) or Takeaway (with a pickup time) when placing an order.
FR-27	The system shall generate a unique order ID, and record the ordered items, total amount, order type, status, and timestamp for every placed order.
FR-28	The system shall automatically progress an order's status from Pending to Preparing, Ready, and Served at fixed time intervals, and display the elapsed time and a progress indicator.

3.7 Manager Dashboard

ID	Requirement
FR-29	The system shall allow the Manager to view all incoming orders and manually change an order's status.
FR-30	The system shall allow the Manager to accept or decline pending reservations.
FR-31	The system shall allow the Manager to add a new menu item, edit an existing item's price, and toggle its availability.
FR-32	The system shall reflect any menu changes made by the Manager immediately on the Customer's Menu screen through shared application state.
FR-33	The system shall persist orders, reservations, and menu edits using AsyncStorage so that data survives an app restart.

Total Functional Requirements: 33

4. Non-Functional Requirements

Usability

- The app shall use a consistent layout, iconography, and color palette across all screens so that navigation feels predictable.
- All interactive elements (buttons, inputs) shall provide immediate visual feedback (e.g. disabled state, loading spinner) when tapped.

Performance

- The menu list shall render smoothly using FlatList's virtualization even as the number of items grows.
- Expensive calculations (cart totals, filtered/sorted lists) shall be memoized using useMemo to avoid unnecessary recalculation on every render.

Responsiveness

- All screens shall adapt to different phone screen sizes using flexbox layouts rather than fixed pixel dimensions.
- The app shall respect device safe areas (notches, status bars) on both Android and iOS.

Maintainability

- Business logic (validation, reservation availability, form handling) shall be isolated into custom hooks, keeping screen components focused on presentation.
- State related to a single concern (cart, auth, theme, orders) shall be managed by its own Context and, where applicable, its own reducer.

Data Handling Without Backend

- All persistent data (orders, reservations, menu edits) shall be stored on-device using AsyncStorage and loaded on app start.
- The app shall show a loading screen while AsyncStorage data is being read on startup, so the UI never renders with incomplete state.

5. Client-Side Data Model

The following entities model all data used by the application. Since there is no backend, these structures exist purely as local JavaScript objects/arrays, some of which are persisted via AsyncStorage.

User

Field	Type
id	string
fullName	string
email	string
password	string
role	'customer' | 'manager'

Category

Field	Type
id	string
name	string

MenuItem

Field	Type
id	string
name	string
description	string
price	number
category	string
image	string (URI)
isSpecial	boolean
isAvailable	boolean

Table

Field	Type
id	string
tableNumber	number
capacity	number

Field	Type
isAvailable	boolean

Reservation

Field	Type
id	string
userId	string
date	string (YYYY-MM-DD)
time	string (HH:mm)
partySize	number
tableId	string
contactPhone	string
status	'pending' | 'accepted' | 'declined' | 'cancelled'

Order

Field	Type
id	string
userId	string
items	CartItem[]
total	number
type	'dine-in' | 'takeaway'
tableId / pickupTime	string
status	'pending' | 'preparing' | 'ready' | 'served' | 'cancelled'
timestamp	number

6. UML Diagrams

The following UML diagrams model the structure and behaviour of the Restaurant App MVP. The diagram files themselves (exported as PNG/PDF) are provided separately inside the **A1/UML/** folder of this submission, as required by the assignment. They are summarized here for reference:

- **Use Case Diagram** — models the Customer and Restaurant Manager actors against at least 12 use cases (e.g. Sign Up, Browse Menu, Search Menu, Add to Cart, Apply Promo, Reserve Table, Place Order, Track Order, Manage Orders, Manage Reservations, Manage Menu), including relevant <<include>> and <<extend>> relationships.
- **Class Diagram** — models User, Customer, Manager, Category, MenuItem, Cart, CartItem, Order, Reservation, and Table, including attributes with types, key operations, inheritance (Customer/Manager extend User), and multiplicities between related classes.
- **Sequence Diagram** — models the “Customer places an order” scenario, starting at Add to Cart and ending at Order Tracking, showing interaction between the Customer, CartContext, OrdersContext, and the Order Tracking screen.
- **State Machine Diagram** — models the lifecycle of an Order: Pending → Preparing → Ready → Served, with a Cancelled state reachable from Pending or Preparing.
- **Component Diagram** — models the React Native screens, Context providers, reducers, and custom hooks as components, showing their dependencies on one another.

7. MVP Frontend Development

This section describes each of the eight core screens of the MVP: their purpose, key UI elements, navigation behaviour, local data used, and the React hooks each screen relies on.

7.1 Login / Signup Screen

Purpose	Authenticates an existing user or registers a new one, and routes the user to the correct home screen based on their role.
UI Elements	Toggle between Login/Signup mode, email input, password input with show/hide toggle, full name and confirm-password inputs (signup only), role selector (Customer/Manager), submit button with ActivityIndicator while submitting.
Navigation	On success, a Customer is routed to the Menu screen and a Manager to the Manager Dashboard, resetting the navigation stack so the back button cannot return to Login.
Local Data	A mock array of users in src/data/users.js used to validate credentials.
Hooks Used	useState for form fields, mode, errors, show-password, and isSubmitting.

7.2 Menu Screen

Purpose	Lets the customer browse all available menu items grouped by category.
UI Elements	Category filter tabs, FlatList of MenuItemCard components, Daily Special badge, disabled Add button on unavailable items, header showing item count, pull-to-refresh.
Navigation	Accessible from the bottom tab bar; tapping an item's Add button updates the Cart badge in the tab bar.
Local Data	A mock array of 15+ items in src/data/menu.js.
Hooks Used	useState for loading/error/category-filter state, useEffect to simulate the initial data fetch and clean up on unmount.

7.3 Search Screen

Purpose	Lets the customer quickly find a specific menu item by name.
UI Elements	Search input with a focus/clear control, FlatList of filtered results, a Back-to-Top button, recent-searches list.
Navigation	Accessible from the Menu screen header; returns to Menu on back.
Local Data	Filters the same mock menu array used by the Menu screen.
Hooks Used	useRef for the search input, the FlatList reference, the debounce timer, and a render counter.

7.4 Cart Screen

Purpose	Lets the customer review, adjust, and check out the items they intend to order.
UI Elements	List of cart items with quantity steppers, notes field, promo code input, subtotal/discount/total summary, Checkout button.

Navigation	Accessible from the bottom tab bar (with a badge showing item count); Checkout navigates to Order placement.
Local Data	None external — state lives entirely in CartContext.
Hooks Used	useReducer via CartContext for all cart mutations (ADD_ITEM, REMOVE_ITEM, INCREMENT, DECREMENT, UPDATE_NOTE, CLEAR_CART, APPLY_PROMO, REMOVE_PROMO).

7.5 Reservation Screen

Purpose	Lets the customer book a table for a given date, time, and party size.
UI Elements	Date picker, time-slot grid (disabling unavailable/past slots), party-size stepper, table selector, contact fields, confirmation modal, My Reservations list.
Navigation	Accessible from the bottom tab bar; confirming a reservation shows a modal and stays on-screen.
Local Data	A mock array of tables in src/data/tables.js and reservations persisted via AsyncStorage.
Hooks Used	Business logic lives in the custom hooks useForm, useDebounce, and useReservation — the screen itself contains only UI.

7.6 Order Tracking Screen

Purpose	Shows the customer the live status of their most recent order.
UI Elements	Progress indicator across Pending/Preparing/Ready/Served, elapsed time counter, order summary.
Navigation	Shown automatically after placing an order; also reachable from a persistent “Track Order” entry point.
Local Data	The active order object from OrdersContext, persisted via AsyncStorage.
Hooks Used	useEffect with setInterval to advance status automatically, with proper cleanup on unmount.

7.7 Profile Screen

Purpose	Shows the logged-in user's details and lets them switch app theme or log out.
UI Elements	User name/email/role display, light/dark theme toggle, Logout button.
Navigation	Logout resets the navigation stack back to the Login screen.
Local Data	The current user object from AuthContext.
Hooks Used	useAuth and useTheme (custom hooks built on useContext) for login/logout state and theme palette.

7.8 Manager Dashboard Screen

Purpose	Gives the Manager a single place to manage orders, reservations, and the menu.

UI Elements	Three tabs — Incoming Orders (status-change controls), Reservations (accept/decline), Menu Management (add item, edit price, toggle availability).
Navigation	Shown as the Manager's home screen after login, in place of the Customer's Menu/Cart/Reservation tabs.
Local Data	Shared OrdersContext, reservation data, and menu data — all persisted via AsyncStorage so changes survive a restart.
Hooks Used	useReducer (OrdersContext) for order status changes; useCallback for stable handler references passed into memoized list rows; React.memo on row components to avoid unnecessary re-renders.